

SECTION – A

Q1 (a) Answer ANY THREE

1. Define Data Structure

A **Data Structure** is a way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently.

Example: Array, Stack, Queue, Tree, Graph.

// Example of Array as a Data Structure

```
int arr[5] = {10, 20, 30, 40, 50};
```

2. What is Stack?

A **Stack** is a linear data structure that follows the principle **LIFO (Last In First Out)**. The element inserted last is removed first.

Basic Operations:

- Push (insert)
- Pop (delete)
- Peek (view top element)

```
#include <iostream>
```

```
using namespace std;
```

```
int stack[5], top = -1;
```

```
void push(int x) {  
    stack[++top] = x;  
}
```

```
void pop() {
```

```
    top--;  
}
```

3. Define Queue

A **Queue** is a linear data structure that follows **FIFO (First In First Out)** principle. Insertion happens at **rear**, deletion at **front**.

```
#include <iostream>  
  
using namespace std;  
  
int queue[5], front = 0, rear = -1;  
  
void enqueue(int x) {  
    queue[++rear] = x;  
}  
  
void dequeue() {  
    front++;  
}
```

4. What is a Pointer?

A **Pointer** is a variable that stores the **address of another variable**.

```
int a = 10;  
  
int *p = &a; // p stores address of a
```

Q1 (b) Answer ANY THREE

1. Define Binary Tree

A **Binary Tree** is a non-linear data structure in which each node has **at most two children**, called **left child** and **right child**.

```
struct Node {
```

```
int data;

Node* left;

Node* right;

};
```

2. What is Hashing?

Hashing is a technique used to map data to a fixed-size table using a **hash function** for fast searching and insertion.

```
int hashFunction(int key) {

    return key % 10;

}
```

3. Define Graph

A **Graph** is a non-linear data structure consisting of **vertices (nodes)** and **edges** connecting them.

// Adjacency Matrix representation

```
int graph[3][3] = {

    {0, 1, 1},

    {1, 0, 0},

    {1, 0, 0}

};
```

4. What is Recursion?

Recursion is a process where a function calls itself until a base condition is met.

```
int factorial(int n) {

    if (n == 0)

        return 1;

    return n * factorial(n - 1);

}
```

Q2 – 9 Marks

a) Explain Stack Operations with Diagram (4 Marks)

Stack Operations

1. **Push** – Insert an element at the top
2. **Pop** – Remove the top element
3. **Peek** – View the top element

Diagram:

| 30 | ← Top

| 20 |

| 10 |

C++ Implementation

```
#include <iostream>
```

```
using namespace std;
```

```
int stack[5], top = -1;
```

```
void push(int x) {
```

```
    if (top == 4)
```

```
        cout << "Stack Overflow\n";
```

```
    else
```

```
        stack[++top] = x;
```

```
}
```

```
void pop() {
```

```
    if (top == -1)
```

```
        cout << "Stack Underflow\n";
```

```
else  
    top--;  
}
```

b) Answer ANY ONE (5 Marks)

Evaluate Postfix Expression: 5 3 + 8 2 - *

Step-by-Step Evaluation:

1. $5\ 3\ + \rightarrow 8$
2. $8\ 2\ - \rightarrow 6$
3. $8\ 6\ * \rightarrow 48$

C++ Code

```
#include <iostream>  
  
#include <stack>  
  
using namespace std;  
  
int main() {  
    stack<int> s;  
  
    s.push(5);  
    s.push(3);  
    int a = s.top(); s.pop();  
    int b = s.top(); s.pop();  
    s.push(a + b); // 8  
  
    s.push(8);  
    s.push(2);  
    a = s.top(); s.pop();  
    b = s.top(); s.pop();  
    s.push(b - a); // 6
```

```
a = s.top(); s.pop();  
b = s.top(); s.pop();  
s.push(a * b); // 48  
  
cout << "Result = " << s.top();  
return 0;  
}
```

Q3 – 9 Marks

a) Explain Linear Search Algorithm (4 Marks)

Linear Search checks each element of the list one by one until the desired element is found.

Algorithm Steps

1. Start from first element
2. Compare with target
3. If match found → stop
4. Else move to next element

C++ Code

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    int arr[5] = {10, 20, 30, 40, 50};  
    int key = 30;  
  
    for (int i = 0; i < 5; i++) {  
        if (arr[i] == key) {  
            cout << "Element found at index " << i;
```

```
        break;
    }
}
return 0;
}
```

b) Answer ANY ONE (5 Marks)

Explain Binary Search with Example

Binary Search works on **sorted arrays** by repeatedly dividing the search space into half.

Steps

1. Find middle element
2. Compare with key
3. If $\text{key} < \text{mid}$ → search left
4. If $\text{key} > \text{mid}$ → search right

C++ Code

```
#include <iostream>
```

```
using namespace std;
```

```
int binarySearch(int arr[], int n, int key) {
```

```
    int low = 0, high = n - 1;
```

```
    while (low <= high) {
```

```
        int mid = (low + high) / 2;
```

```
        if (arr[mid] == key)
```

```
            return mid;
```

```
        else if (arr[mid] < key)
```

```
            low = mid + 1;
```

```
        else
```

```
        high = mid - 1;
    }
    return -1;
}

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int key = 40;
    cout << binarySearch(arr, 5, key);
    return 0;
}
```


SECTION – B

Q4 (Compulsory) – 12 Marks

Q4 (a) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Linked List

A **Linked List** is a linear data structure in which elements (nodes) are stored in non-contiguous memory locations.

Each node contains:

- **Data**
- **Address of next node**

```
struct Node {  
    int data;  
    Node* next;  
};
```

2. What is Circular Queue?

A **Circular Queue** is a queue in which the last position is connected back to the first position to make efficient use of memory.

$\text{rear} = (\text{rear} + 1) \% \text{size};$

3. Define AVL Tree

An **AVL Tree** is a **self-balancing Binary Search Tree** where the height difference between left and right subtrees is at most **1**.

$\text{Balance Factor} = \text{Height}(\text{Left}) - \text{Height}(\text{Right})$

4. What is BFS?

BFS (Breadth First Search) is a graph traversal algorithm that explores nodes **level by level** using a **queue**.

// BFS uses Queue data structure

Q4 (b) Answer ANY THREE (3 × 2 = 6 Marks)

1. Define BST

A **Binary Search Tree (BST)** is a binary tree in which:

- Left child < Root
- Right child > Root

if (key < root->data)

root->left = insert(root->left, key);

2. What is Collision?

A **Collision** occurs in hashing when **two different keys produce the same hash value**.

hash(key1) == hash(key2)

3. Define DFS

DFS (Depth First Search) is a graph traversal technique that explores as far as possible along a branch before backtracking.

It uses **stack or recursion**.

// DFS uses Stack or Recursion

4. What is Deque?

A **Deque (Double Ended Queue)** allows insertion and deletion from **both front and rear ends**.

// Insertion and deletion from both ends

Q5 – 9 Marks

a) Explain Singly Linked List Insertion (4 Marks)

Insertion in Singly Linked List

Insertion can be done:

1. At Beginning

2. At End
3. At Specific Position

Insertion at Beginning – C++ Code

```
#include <iostream>

using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* insertAtBeginning(Node* head, int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = head;
    return newNode;
}
```

Insertion at End – C++ Code

```
Node* insertAtEnd(Node* head, int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;
```

```
Node* temp = head;
while (temp->next != NULL)
    temp = temp->next;

temp->next = newNode;
return head;
}
```

b) Answer ANY ONE (5 Marks)

Explain Doubly Linked List with Diagram

A **Doubly Linked List** is a linked list in which each node contains:

- Previous pointer
- Data
- Next pointer

Diagram

NULL <- [10] <-> [20] <-> [30] -> NULL

C++ Structure

```
struct Node {
    int data;
    Node* prev;
    Node* next;
};
```

Advantages

- Traversal in both directions
 - Easy deletion
-

Q6 – 9 Marks

a) Explain Binary Tree Traversals (4 Marks)

Binary Tree traversal means visiting each node **exactly once**.

Types of Traversals

1. Inorder (LNR)
 2. Preorder (NLR)
 3. Postorder (LRN)
-

C++ Code

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};  
  
void inorder(Node* root) {  
    if (root != NULL) {  
        inorder(root->left);  
        cout << root->data << " ";  
        inorder(root->right);  
    }  
}  
  
void preorder(Node* root) {  
    if (root != NULL) {  
        cout << root->data << " ";  
        preorder(root->left);  
        preorder(root->right);  
    }  
}
```

```

void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}

```

b) Answer ANY ONE (5 Marks)

Explain Binary Search Tree (BST)

A **Binary Search Tree** is a binary tree where:

- Left subtree contains smaller values
 - Right subtree contains larger values
-

BST Insertion – C++ Code

```

Node* insert(Node* root, int key) {
    if (root == NULL) {
        Node* newNode = new Node();
        newNode->data = key;
        newNode->left = newNode->right = NULL;
        return newNode;
    }

    if (key < root->data)
        root->left = insert(root->left, key);
    else
        root->right = insert(root->right, key);
}

```

```
return root;  
}
```

Advantages of BST

- Fast searching
- Easy insertion and deletion

Paper 2

SECTION – A

Q1 (Compulsory) – 12 Marks

Q1 (a) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. What is Algorithm?

An **Algorithm** is a finite set of well-defined steps used to solve a specific problem.

An algorithm must be:

- Clear
- Finite
- Correct

// Algorithm to add two numbers

```
int sum(int a, int b) {  
    return a + b;  
}
```

2. Define Dynamic Memory Allocation

Dynamic Memory Allocation is the process of allocating memory at **runtime** using pointers.

In C++, it is done using new and delete.

```
int* p = new int; // allocate memory  
  
*p = 10;  
  
delete p; // free memory
```

3. What is Stack Overflow?

Stack Overflow occurs when we try to **push an element into a stack that is already full**.

```
if (top == size - 1)  
    cout << "Stack Overflow";
```

4. Define Dequeue

A **Deque (Double Ended Queue)** is a linear data structure that allows **insertion and deletion from both ends** (front and rear).

// Deque allows insertion and deletion at both ends

Q1 (b) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Graph

A **Graph** is a non-linear data structure consisting of **vertices (nodes)** and **edges** connecting them.

// Adjacency Matrix representation

```
int graph[3][3] = {  
    {0,1,0},  
    {1,0,1},  
    {0,1,0}  
};
```

2. What is Hash Function?

A **Hash Function** converts a key into an index of a hash table.

```
int hashFunction(int key) {  
    return key % 10;  
}
```

3. Define Recursion

Recursion is a programming technique where a function calls itself until a base condition is met.

```
int factorial(int n) {  
    if (n == 0)  
        return 1;  
    return n * factorial(n - 1);  
}
```

}

4. What is Load Factor?

Load Factor is the ratio of **number of elements stored** to **size of hash table**.

$$\text{Load Factor} = \frac{\text{Number of elements stored}}{\text{Size of hash table}}$$

Q2 – 9 Marks

a) Explain Queue Operations (4 Marks)

A **Queue** follows **FIFO (First In First Out)**.

Queue Operations

1. **Enqueue** – Insert element at rear
2. **Dequeue** – Remove element from front
3. **Peek** – View front element
4. **IsEmpty / IsFull**

C++ Implementation

```
#include <iostream>
```

```
using namespace std;
```

```
int queue[5], front = 0, rear = -1;
```

```
void enqueue(int x) {
```

```
    if (rear == 4)
```

```
        cout << "Queue Overflow\n";
```

```

else
    queue[++rear] = x;
}

void dequeue() {
    if (front > rear)
        cout << "Queue Underflow\n";
    else
        front++;
}

```

b) Answer ANY ONE (5 Marks)

Explain Circular Queue with Diagram

A **Circular Queue** connects the last position to the first position to efficiently use memory.

Diagram

Front → [10] [20] [30] [] ← Rear

↑ _____ ↓

C++ Implementation

```

#include <iostream>

using namespace std;

int cq[5], front = -1, rear = -1, size = 5;

void enqueue(int x) {
    if ((rear + 1) % size == front)
        cout << "Queue Full\n";
    else {
        if (front == -1)

```

```
        front = 0;
        rear = (rear + 1) % size;
        cq[rear] = x;
    }
}

void dequeue() {
    if (front == -1)
        cout << "Queue Empty\n";
    else if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % size;
}
```

Q3 – 9 Marks

a) Explain Merge Sort (4 Marks)

Merge Sort is a **divide and conquer** sorting algorithm.

Steps

1. Divide array into two halves
2. Recursively sort each half
3. Merge the sorted halves

Characteristics

- Stable sorting
 - Uses recursion
 - Time Complexity: $O(n \log n)$
-

C++ Code

```
#include <iostream>
```

```
using namespace std;
```

```
void merge(int arr[], int l, int m, int r) {
```

```
    int i = l, j = m + 1;
```

```
    int temp[10], k = l;
```

```
    while (i <= m && j <= r) {
```

```
        if (arr[i] < arr[j])
```

```
            temp[k++] = arr[i++];
```

```
        else
```

```
            temp[k++] = arr[j++];
```

```
    }
```

```
    while (i <= m)
```

```
        temp[k++] = arr[i++];
```

```
    while (j <= r)
```

```
        temp[k++] = arr[j++];
```

```
    for (i = l; i <= r; i++)
```

```
        arr[i] = temp[i];
```

```
}
```

```
void mergeSort(int arr[], int l, int r) {
```

```
    if (l < r) {
```

```
        int m = (l + r) / 2;
```

```
mergeSort(arr, l, m);  
mergeSort(arr, m + 1, r);  
merge(arr, l, m, r);  
}  
}
```

b) Answer ANY ONE (5 Marks)

Sort using Merge Sort: 67, 25, 89, 12

Step 1: Divide

67, 25, 89, 12

→ [67,25] [89,12]

Step 2: Sort sub-arrays

[25,67] [12,89]

Step 3: Merge

[12, 25, 67, 89]

Final Sorted Array

12 25 67 89

SECTION – B

Q4 (Compulsory) – 12 Marks

Q4 (a) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Tree

A **Tree** is a **non-linear data structure** consisting of nodes connected by edges.

It has:

- One **root**
- Zero or more **child nodes**
- No cycles

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

2. What is Balance Factor?

Balance Factor is used in AVL trees to check whether the tree is balanced.

$$\text{Balance Factor} = \text{Height of Left Subtree} - \text{Height of Right Subtree}$$

- Valid values: **-1, 0, +1**
-

3. Define Hash Table

A **Hash Table** is a data structure that stores data in an array format using a **hash function** to compute index values.

```
int hashTable[10];
```

4. What is DFS?

DFS (Depth First Search) is a graph traversal technique that explores as deep as possible along a branch before backtracking.

It uses **stack or recursion**.

// DFS uses stack or recursion

Q4 (b) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define BFS

BFS (Breadth First Search) is a graph traversal algorithm that visits nodes **level by level**.

It uses a **queue**.

// BFS uses queue

2. What is BST?

A **Binary Search Tree (BST)** is a binary tree where:

- Left child < Root
- Right child > Root

if (key < root->data)

root->left = insert(root->left, key);

3. Define Adjacency Matrix

An **Adjacency Matrix** is a 2D array used to represent a graph where:

- 1 means edge exists
- 0 means no edge

int graph[3][3] = {

{0,1,0},

{1,0,1},

{0,1,0}

};

4. What is Collision Resolution?

Collision Resolution is the method used to handle collisions in hashing when two keys map to the same index.

Techniques:

- Chaining
 - Linear Probing
 - Quadratic Probing
-

Q5 – 9 Marks

a) Explain AVL Tree Rotations (4 Marks)

An **AVL Tree** is a self-balancing BST.

When the balance factor becomes **> 1 or < -1**, rotations are applied.

Types of Rotations

1. **LL Rotation**
 2. **RR Rotation**
 3. **LR Rotation**
 4. **RL Rotation**
-

Example (Right Rotation – LL Case)

```
Node* rightRotate(Node* y) {  
    Node* x = y->left;  
    Node* T2 = x->right;  
  
    x->right = y;  
    y->left = T2;  
  
    return x;  
}
```

b) Answer ANY ONE (5 Marks)

Insert 10, 20, 30 into AVL Tree

Step 1: Insert 10

10

Step 2: Insert 20

10
 \
 20

Step 3: Insert 30 (Unbalanced → RR case)

10
 \
 20
 \
 30

Apply Left Rotation (RR Rotation)

Final AVL Tree

20
 /
10 30

Q6 – 9 Marks

a) Explain BFS Traversal (4 Marks)

BFS (Breadth First Search) traverses a graph or tree **level by level** using a **queue**.

Algorithm Steps

1. Start from root
 2. Visit node
 3. Add its neighbors to queue
 4. Repeat until queue is empty
-

C++ Code for BFS

```
#include <iostream>

#include <queue>

using namespace std;

void BFS(int graph[4][4], int start) {

    bool visited[4] = {false};

    queue<int> q;

    visited[start] = true;

    q.push(start);

    while (!q.empty()) {

        int v = q.front();

        q.pop();

        cout << v << " ";

        for (int i = 0; i < 4; i++) {

            if (graph[v][i] == 1 && !visited[i]) {

                visited[i] = true;

                q.push(i);

            }

        }

    }

}
```

b) Answer ANY ONE (5 Marks)

Explain DFS Traversal

DFS (Depth First Search) explores as deep as possible before backtracking.
It uses **recursion or stack**.

C++ Code for DFS

```
#include <iostream>

using namespace std;

void DFS(int graph[4][4], int v, bool visited[]) {
    visited[v] = true;
    cout << v << " ";

    for (int i = 0; i < 4; i++) {
        if (graph[v][i] == 1 && !visited[i]) {
            DFS(graph, i, visited);
        }
    }
}
```

Advantages of DFS

- Less memory than BFS
- Useful for path finding

SECTION – A (30 Marks)

Q1 (Compulsory) – 12 Marks

Q1 (a) Answer ANY THREE (3 × 2 = 6 Marks)

1. Define Algorithm

An **Algorithm** is a finite sequence of well-defined steps used to solve a specific problem.

An algorithm must be:

- Correct
- Finite
- Unambiguous

Example (C++):

// Algorithm to find maximum of two numbers

```
int max(int a, int b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```

2. What is Stack Overflow?

Stack Overflow occurs when an attempt is made to **push an element into a stack that is already full**.

Condition:

```
if (top == size - 1)  
    cout << "Stack Overflow";
```

3. Define Dynamic Memory Allocation

Dynamic Memory Allocation is the process of allocating memory at **run time** instead of compile time.

In C++, it is done using new and delete.

```
int* ptr = new int; // allocate memory
```

```
*ptr = 10;
```

```
delete ptr; // free memory
```

4. What is a Circular Queue?

A **Circular Queue** is a queue in which the last position is connected back to the first position to make efficient use of memory.

```
rear = (rear + 1) % size;
```

Q1 (b) Answer ANY THREE (3 × 2 = 6 Marks)

1. Define Binary Tree

A **Binary Tree** is a non-linear data structure in which each node has **at most two children**, called left child and right child.

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

2. What is a Hash Function?

A **Hash Function** converts a key into a fixed-size value (index) used in a hash table.

```
int hashFunction(int key) {  
    return key % 10;  
}
```

3. Define Recursion

Recursion is a programming technique where a function calls itself until a base condition is satisfied.

```
int factorial(int n) {  
    if (n == 0)  
        return 1;  
    return n * factorial(n - 1);  
}
```

4. What is Load Factor?

Load Factor is the ratio of the number of elements stored in a hash table to the total size of the table.

$$\text{Load Factor} = \frac{\text{Number of elements}}{\text{Table size}}$$

Q2 – 9 Marks

Q2 (a) Explain Stack Operations with Diagram (4 Marks)

A **Stack** follows the **LIFO (Last In First Out)** principle.

Stack Operations

1. **Push** – Insert element at the top
2. **Pop** – Remove top element
3. **Peek** – View top element

Diagram

| 30 | ← Top

| 20 |

| 10 |

C++ Implementation

```
#include <iostream>
```

```
using namespace std;
```

```
int stack[5], top = -1;
```

```
void push(int x) {
```

```
    if (top == 4)
```

```
        cout << "Stack Overflow\n";
```

```
    else
```

```
        stack[++top] = x;
```

```
}
```

```
void pop() {
```

```
    if (top == -1)
```

```
        cout << "Stack Underflow\n";
```

```
    else
```

```
        top--;
```

```
}
```

Q2 (b) Answer ANY ONE (5 Marks)

Convert infix expression $A + (B * C)$ into postfix

Operator precedence:

- $*$ has higher precedence than $+$

Steps:

Infix: $A + (B * C)$

Postfix: $A B C * +$

 **Final Postfix Expression:**

ABC*+

OR

Evaluate postfix expression: 5 6 2 + *

Step-by-step evaluation:

1. $6\ 2\ + \rightarrow 8$
2. $5\ 8\ * \rightarrow 40$

C++ Code

```
#include <iostream>
```

```
#include <stack>
```

```
using namespace std;
```

```
int main() {
```

```
    stack<int> s;
```

```
    s.push(5);
```

```
    s.push(6);
```

```
    s.push(2);
```

```
    int a = s.top(); s.pop();
```

```
    int b = s.top(); s.pop();
```

```
    s.push(b + a); // 8
```

```
    a = s.top(); s.pop();
```

```
    b = s.top(); s.pop();
```

```
    s.push(b * a); // 40
```

```
    cout << "Result = " << s.top();
```

```
    return 0;
}
```

Q3 – 9 Marks

Q3 (a) Explain Linear Search Algorithm with Example (4 Marks)

Linear Search checks each element of the list one by one until the desired element is found.

Algorithm Steps

1. Start from first element
2. Compare with key
3. If found, stop
4. Else move to next element

Example & C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int key = 30;

    for (int i = 0; i < 5; i++) {
        if (arr[i] == key) {
            cout << "Element found at index " << i;
            break;
        }
    }

    return 0;
}
```

Q3 (b) Answer ANY ONE (5 Marks)

Explain Binary Search Algorithm with Example

Binary Search works only on **sorted arrays**.

It repeatedly divides the search space into half.

Steps

1. Find middle element
2. Compare with key
3. If $\text{key} < \text{middle}$ → left side
4. If $\text{key} > \text{middle}$ → right side

C++ Code

```
#include <iostream>
```

```
using namespace std;
```

```
int binarySearch(int arr[], int n, int key) {
```

```
    int low = 0, high = n - 1;
```

```
    while (low <= high) {
```

```
        int mid = (low + high) / 2;
```

```
        if (arr[mid] == key)
```

```
            return mid;
```

```
        else if (arr[mid] < key)
```

```
            low = mid + 1;
```

```
        else
```

```
            high = mid - 1;
```

```
    }
```

```
    return -1;
```

```
}
```

```
int main() {  
    int arr[5] = {10, 20, 30, 40, 50};  
    int key = 40;  
  
    cout << binarySearch(arr, 5, key);  
    return 0;  
}
```

OR

Explain Interpolation Search

Interpolation Search is an improved version of binary search where the position of the element is estimated.

Formula:

$$pos = low + \frac{(key - arr[low])}{(arr[high] - arr[low])} \times (high - low)$$

It works best when data is **uniformly distributed**.

SECTION – B (30 Marks)

Q4 (Compulsory) – 12 Marks

Q4 (a) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Linked List

A **Linked List** is a linear data structure in which elements are stored in **nodes**.

Each node contains:

- **Data**
- **Address of next node**

Nodes are not stored in continuous memory locations.

```
struct Node {
```

```
    int data;
```

```
    Node* next;
```

```
};
```

2. What is a Deque?

A **Deque (Double Ended Queue)** is a queue in which insertion and deletion can be done from **both front and rear ends**.

Types:

- Input restricted deque
- Output restricted deque

// Deque allows insertion and deletion at both ends

3. Define AVL Tree

An **AVL Tree** is a **self-balancing Binary Search Tree** in which the difference between the heights of left and right subtrees is at most **1**.

Balance Factor = Height(left) - Height(right)

4. What is Breadth First Search (BFS)?

BFS is a graph traversal algorithm that visits nodes **level by level**.
It uses a **queue** data structure.

// BFS uses Queue

Q4 (b) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Binary Search Tree (BST)

A **Binary Search Tree** is a binary tree in which:

- Left subtree contains values smaller than root
- Right subtree contains values greater than root

if (key < root->data)

root->left = insert(root->left, key);

2. What is Collision in Hashing?

A **Collision** occurs when **two different keys produce the same hash value** and try to occupy the same index in a hash table.

hash(key1) == hash(key2)

3. Define Depth First Search (DFS)

DFS is a graph traversal technique that explores a node and then goes as deep as possible before backtracking.

It uses **stack or recursion**.

// DFS uses recursion or stack

4. What is an Adjacency Matrix?

An **Adjacency Matrix** is a 2D array used to represent a graph.
If there is an edge between two vertices, value is **1**, otherwise **0**.

```
int graph[3][3] = {  
    {0, 1, 0},
```

```
{1, 0, 1},  
{0, 1, 0}  
};
```

Q5 – 9 Marks

Q5 (a) Explain insertion at beginning in a Singly Linked List with diagram (4 Marks)

Insertion at Beginning

A new node is created and inserted before the first node.
The new node becomes the **head** of the list.

Steps

1. Create a new node
2. Assign data to it
3. Point new node's next to current head
4. Update head

Diagram

Before:

Head → [10] → [20] → NULL

After inserting 5:

Head → [5] → [10] → [20] → NULL

C++ Code

```
#include <iostream>  
using namespace std;
```

```
struct Node {  
    int data;  
    Node* next;
```

```
};
```

```
Node* insertAtBeginning(Node* head, int value) {  
    Node* newNode = new Node();  
    newNode->data = value;  
    newNode->next = head;  
    return newNode;  
}
```

Q5 (b) Answer ANY ONE (5 Marks)

Explain Doubly Linked List with Diagram

A **Doubly Linked List** is a linked list where each node contains:

- Address of previous node
- Data
- Address of next node

Diagram

NULL ← [10] ⇌ [20] ⇌ [30] → NULL

C++ Structure

```
struct Node {  
    int data;  
    Node* prev;  
    Node* next;  
};
```

Advantages

- Traversal in both directions
 - Easy deletion of nodes
-

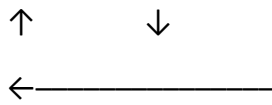
OR

Explain Circular Linked List with Suitable Example

In a **Circular Linked List**, the last node points back to the first node instead of NULL.

Diagram

[10] → [20] → [30]



C++ Structure

```
struct Node {  
    int data;  
    Node* next;  
};
```

Example Use

- CPU scheduling
- Multiplayer games

Q6 – 9 Marks

Q6 (a) Explain Binary Tree Traversals (4 Marks)

Traversal means visiting each node of a binary tree **exactly once**.

Types of Traversals

1. **Inorder (LNR)**
Left → Node → Right
2. **Preorder (NLR)**
Node → Left → Right
3. **Postorder (LRN)**
Left → Right → Node

C++ Code

```
struct Node {
```

```
int data;

Node* left;

Node* right;
};

void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

void preorder(Node* root) {
    if (root != NULL) {
        cout << root->data << " ";
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}
```

Q6 (b) Answer ANY ONE (5 Marks)

Explain Binary Search Tree and its Properties

A **Binary Search Tree (BST)** is a binary tree where:

- Left subtree contains smaller values
- Right subtree contains larger values
- No duplicate values

Properties

1. Inorder traversal gives sorted order
2. Searching is efficient
3. Average time complexity: **$O(\log n)$**

C++ Insertion Code

```
Node* insert(Node* root, int key) {  
    if (root == NULL) {  
        Node* newNode = new Node();  
        newNode->data = key;  
        newNode->left = newNode->right = NULL;  
        return newNode;  
    }  
  
    if (key < root->data)  
        root->left = insert(root->left, key);  
    else  
        root->right = insert(root->right, key);  
  
    return root;  
}
```

OR

Explain Expression Tree with Example

An **Expression Tree** is a binary tree used to represent arithmetic expressions.

Example

Expression:

$(A + B) * C$

Expression Tree

*

/ \

+ C

/ \

A B

Uses

- Expression evaluation
- Compiler design

SECTION – A (30 Marks)

Q1 (Compulsory) – 12 Marks

Q1 (a) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Time Complexity

Time Complexity is a measure of the **amount of time an algorithm takes to run** as a function of the input size.

It is expressed using **Big-O notation**.

Example:

```
// Time Complexity: O(n)
for (int i = 0; i < n; i++) {
    cout << i;
}
```

2. What is Stack Underflow?

Stack Underflow occurs when we try to **remove (pop)** an element from an empty stack.

```
if (top == -1)
    cout << "Stack Underflow";
```

3. Define Recursion

Recursion is a programming technique where a function **calls itself** until a **base condition** is satisfied.

```
int factorial(int n) {
    if (n == 0)
        return 1;
    return n * factorial(n - 1);
}
```

```
}
```

4. What is a Priority Queue?

A **Priority Queue** is a special type of queue in which each element is associated with a **priority**, and elements are removed based on priority rather than insertion order.

// Higher priority element is removed first

Q1 (b) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Binary Search Tree (BST)

A **Binary Search Tree** is a binary tree in which:

- Left child < Root
- Right child > Root

if (key < root->data)

root->left = insert(root->left, key);

2. What is Balance Factor?

Balance Factor is used in AVL trees to maintain balance.

$$\text{Balance Factor} = \text{Height of Left Subtree} - \text{Height of Right Subtree}$$

Valid values are **-1, 0, +1**.

3. Define Hash Function

A **Hash Function** maps a key to an index in a hash table for fast data access.

```
int hashFunction(int key) {  
    return key % 10;  
}
```

4. What is Load Factor?

Load Factor indicates how full a hash table is.

$$\text{Load Factor} = \frac{\text{Number of elements}}{\text{Table size}}$$

Q2 – 9 Marks

Q2 (a) Explain Recursion using Tower of Hanoi problem (4 Marks)

The **Tower of Hanoi** is a classic problem that demonstrates recursion.

Rules

1. Only one disk can be moved at a time
2. A larger disk cannot be placed on a smaller disk
3. Only one disk can be moved directly

Recursive Idea

- Move n-1 disks from source to auxiliary
 - Move largest disk to destination
 - Move n-1 disks from auxiliary to destination
-

C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
void towerOfHanoi(int n, char source, char auxiliary, char destination) {  
    if (n == 1) {  
        cout << "Move disk 1 from " << source << " to " << destination << endl;  
        return;  
    }  
    towerOfHanoi(n - 1, source, destination, auxiliary);  
    cout << "Move disk " << n << " from " << source << " to " << destination << endl;  
    towerOfHanoi(n - 1, auxiliary, source, destination);  
}
```

}

Q2 (b) Answer ANY ONE (5 Marks)

Write Recursive Algorithm for Factorial and Explain

Factorial of n is defined as:

$$n! = n \times (n - 1)!$$

C++ Program

```
int factorial(int n) {  
    if (n == 0)  
        return 1;  
    return n * factorial(n - 1);  
}
```

Explanation

- Base case: $n == 0$
 - Recursive call reduces the problem size
 - Function keeps calling itself until base condition is met
-

OR

Write Recursive Algorithm for Fibonacci Series

C++ Program

```
int fibonacci(int n) {  
    if (n == 0)  
        return 0;  
    if (n == 1)  
        return 1;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

Q3 – 9 Marks

Q3 (a) Explain Quick Sort Algorithm (4 Marks)

Quick Sort is a **divide and conquer** sorting algorithm.

Steps

1. Select a **pivot element**
 2. Partition the array so that:
 - Elements smaller than pivot are on left
 - Elements greater than pivot are on right
 3. Recursively apply quick sort to left and right sub-arrays
-

C++ Code

```
#include <iostream>

using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
```

```
}
```

```
void quickSort(int arr[], int low, int high) {
```

```
    if (low < high) {
```

```
        int pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
```

```
    }
```

```
}
```

Q3 (b) Answer ANY ONE (5 Marks)

Sort the array using Quick Sort: 45, 12, 78, 34, 23

Initial Array:

45, 12, 78, 34, 23

Pivot = 23

After partition:

12, 23, 78, 34, 45

Left Sub-array: [12] (already sorted)

Right Sub-array: [78, 34, 45]

Pivot = 45

Final sorted array:

12, 23, 34, 45, 78

OR

Explain Time Complexity of Quick Sort

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$

Worst Case	$O(n^2)$
------------	----------

Worst case occurs when pivot is smallest or largest element repeatedly.

SECTION – B (30 Marks)

Q4 (Compulsory) – 12 Marks

Q4 (a) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Linked List

A **Linked List** is a linear data structure in which elements are stored in **nodes**.

Each node contains:

- **Data**
- **Pointer to the next node**

Nodes are not stored in continuous memory locations.

```
struct Node {
```

```
    int data;
```

```
    Node* next;
```

```
};
```

2. What is a Circular Queue?

A **Circular Queue** is a queue where the last position is connected back to the first position, forming a circle.

It improves memory utilization.

```
rear = (rear + 1) % size;
```

3. Define AVL Tree

An **AVL Tree** is a **self-balancing Binary Search Tree** in which the height difference between left and right subtrees is at most **1**.

Balance Factor = Height(left) - Height(right)

4. What is Breadth First Search (BFS)?

BFS is a graph traversal algorithm that visits nodes **level by level** using a **queue**.

// BFS uses Queue data structure

Q4 (b) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Binary Tree

A **Binary Tree** is a non-linear data structure in which each node has **at most two children**.

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

2. What is Collision in Hashing?

A **Collision** occurs when two different keys produce the same hash value and map to the same index.

```
hash(key1) == hash(key2)
```

3. Define Depth First Search (DFS)

DFS is a graph traversal method that explores as deep as possible before backtracking. It uses **recursion or stack**.

// DFS uses stack or recursion

4. What is an Adjacency List?

An **Adjacency List** represents a graph using an array of linked lists. Each vertex stores a list of its adjacent vertices.

```
vector<int> adj[5]; // adjacency list
```

Q5 – 9 Marks

Q5 (a) Explain insertion and deletion operations on Singly Linked List (4 Marks)

Insertion in Singly Linked List

Insertion can be done:

1. At beginning
2. At end

Insertion at Beginning

```
Node* insertAtBeginning(Node* head, int value) {  
    Node* newNode = new Node();  
    newNode->data = value;  
    newNode->next = head;  
    return newNode;  
}
```

Deletion from Beginning

```
Node* deleteAtBeginning(Node* head) {  
    if (head == NULL)  
        return NULL;  
    Node* temp = head;  
    head = head->next;  
    delete temp;  
    return head;  
}
```

Diagram

Before: Head → [10] → [20] → [30] → NULL

After Insertion: Head → [5] → [10] → [20] → [30] → NULL

After Deletion: Head → [10] → [20] → [30] → NULL

Q5 (b) Answer ANY ONE (5 Marks)

Explain Doubly Linked List with Suitable Diagram

A **Doubly Linked List** contains nodes with:

- Previous pointer
- Data
- Next pointer

Diagram

NULL $\leftarrow$ [10] $\rightleftarrows$ [20] $\rightleftarrows$ [30] $\rightarrow$ NULL

C++ Structure

```
struct Node {  
    int data;  
    Node* prev;  
    Node* next;  
};
```

Advantages

- Traversal in both directions
- Easy deletion

OR

Explain Header Linked List and its Advantages

A **Header Linked List** is a linked list that contains a **special header node** at the beginning. The header node does not store actual data.

Types

1. Grounded header list
2. Circular header list

Advantages

- Simplifies insertion and deletion
- No special case for empty list
- Stores metadata (like count of nodes)

Q6 – 9 Marks

Q6 (a) Explain AVL Tree Rotations (4 Marks)

When an AVL tree becomes unbalanced, rotations are applied.

Types of Rotations

1. **LL Rotation** – Right rotation
 2. **RR Rotation** – Left rotation
 3. **LR Rotation**
 4. **RL Rotation**
-

C++ Example (Right Rotation)

```
Node* rightRotate(Node* y) {  
    Node* x = y->left;  
    Node* T2 = x->right;  
  
    x->right = y;  
    y->left = T2;  
  
    return x;  
}
```

Q6 (b) Answer ANY ONE (5 Marks)

Insert 30, 20, 40, 10, 25 into AVL Tree

Step 1: Insert 30

30

Step 2: Insert 20

30

/

20

Step 3: Insert 40

30

/ \

20 40

Step 4: Insert 10

30

/ \

20 40

/

10

Balanced (BF = +1)

Step 5: Insert 25

30

/ \

20 40

/ \

10 25

Tree remains balanced → **No rotation required**

OR

Explain Applications of AVL Tree

1. Database indexing
2. File systems
3. Searching applications
4. Memory management

SECTION – A (30 Marks)

Q1 (Compulsory) – 12 Marks

Q1 (a) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Algorithm

An **Algorithm** is a **step-by-step procedure** for solving a problem in a finite number of steps.

Properties:

- Finite
- Correct
- Well-defined

Example (C++):

// Algorithm to find maximum of two numbers

```
int max(int a, int b){  
    if(a > b) return a;  
    else return b;  
}
```

2. What is Stack Overflow?

Stack Overflow occurs when we try to **push an element onto a stack that is already full**.

C++ Example:

```
int stack[5], top = -1;  
void push(int x){  
    if(top == 4) cout << "Stack Overflow";  
    else stack[++top] = x;  
}
```

3. Define Dynamic Memory Allocation

Dynamic Memory Allocation allocates memory **at runtime** instead of compile-time.

C++ Example:

```
int* ptr = new int; // allocate memory  
  
*ptr = 10;  
  
delete ptr;      // deallocate memory
```

4. What is a Deque?

A **Deque (Double Ended Queue)** allows **insertion and deletion at both front and rear ends**.

// Insertion at front and rear

// Deletion at front and rear

Q1 (b) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Binary Tree

A **Binary Tree** is a non-linear data structure in which each node has **at most two children**.

```
struct Node{  
    int data;  
    Node* left;  
    Node* right;  
};
```

2. What is Hashing?

Hashing maps data to a **fixed-size index** in a hash table using a **hash function**.

```
int hashFunction(int key){  
    return key % 10; // index between 0-9  
}
```

3. Define Graph

A **Graph** is a collection of **vertices (nodes)** and **edges** that connect pairs of vertices.

- **Directed/Undirected**
- **Weighted/Unweighted**

```
struct Graph{  
    int V;  
    vector<int> adjList[];  
};
```

4. What is Recursion Stack?

The **Recursion Stack** is a stack that stores information about **active function calls** in recursion, including parameters and return addresses.

Q2 – 9 Marks

Q2 (a) Explain Evaluation of Postfix Expression Using Stack (4 Marks)

Postfix Expression: Operators come **after operands**.

Algorithm (Stack Method):

1. Initialize an empty stack.
2. Scan the expression from left to right.
3. If operand → push onto stack.
4. If operator → pop two operands, apply operator, push result.
5. After scanning, the stack contains the result.

C++ Code Example:

```
#include <iostream>  
  
#include <stack>  
  
using namespace std;
```

```

int evaluatePostfix(string exp){
    stack<int> s;
    for(char c : exp){
        if(isdigit(c)) s.push(c - '0');
        else{
            int b = s.top(); s.pop();
            int a = s.top(); s.pop();
            if(c == '+') s.push(a+b);
            else if(c == '-') s.push(a-b);
            else if(c == '*') s.push(a*b);
            else if(c == '/') s.push(a/b);
        }
    }
    return s.top();
}

int main(){
    string exp = "623+-3*"; // 6 2 3 + - 3 *
    cout << "Result: " << evaluatePostfix(exp);
}

```

Q2 (b) Answer ANY ONE (5 Marks)

Option 1: Evaluate 6 2 3 + - 3 *

Stepwise Evaluation:

1. Stack: 6
2. Stack: 6, 2
3. Stack: 6, 2, 3
4. + → pop 2 & 3 → 5 → push 5 → Stack: 6, 5

5. $- \rightarrow \text{pop } 6 \ \& \ 5 \rightarrow 1 \rightarrow \text{push } 1 \rightarrow \text{Stack: } 1$

6. $3 \rightarrow \text{push} \rightarrow \text{Stack: } 1, 3$

7. $* \rightarrow \text{pop } 1 \ \& \ 3 \rightarrow 3 \rightarrow \text{push} \rightarrow \text{Stack: } 3$

Result = 3

Option 2: Convert $(A+B)*(C-D)$ to Postfix

Infix: $(A + B) * (C - D)$

Postfix: $A B + C D - *$

Steps:

1. Parentheses handled first
 2. Operators placed after operands
 3. Apply precedence rules
-

Q3 – 9 Marks

Q3 (a) Explain Merge Sort Algorithm (4 Marks)

Merge Sort: Divide and Conquer algorithm for sorting.

Steps:

1. Divide array into two halves
2. Recursively sort left and right halves
3. Merge the sorted halves

C++ Code:

```
void merge(int arr[], int l, int m, int r){
    int n1 = m-l+1, n2 = r-m;
    int L[n1], R[n2];
    for(int i=0;i<n1;i++) L[i]=arr[l+i];
    for(int i=0;i<n2;i++) R[i]=arr[m+1+i];
    int i=0,j=0,k=l;
```

```

while(i<n1 && j<n2){
    if(L[i]<=R[j]) arr[k++]=L[i++];
    else arr[k++]=R[j++];
}
while(i<n1) arr[k++]=L[i++];
while(j<n2) arr[k++]=R[j++];
}

void mergeSort(int arr[], int l, int r){
    if(l<r){
        int m = l+(r-l)/2;
        mergeSort(arr,l,m);
        mergeSort(arr,m+1,r);
        merge(arr,l,m,r);
    }
}

```

Q3 (b) Answer ANY ONE (5 Marks)

Option 1: Sort 67, 25, 89, 12, 55 using Merge Sort

Step 1: Divide

[67,25,89,12,55] → [67,25] & [89,12,55]

Step 2: Recursively sort halves

- [67,25] → [67], [25] → merge → [25,67]
- [89,12,55] → [89], [12,55] → [12,55] → merge → [12,55,89]

Step 3: Merge

[25,67] & [12,55,89] → [12,25,55,67,89]

Sorted Array:

12, 25, 55, 67, 89

Option 2: Explain Time and Space Complexity

Complexity	Merge Sort
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$
Space Complexity	$O(n)$

- **Reason:** Divides array in halves and merges $\rightarrow \log n$ levels $\times n$ elements
- **Space:** Temporary arrays used during merging

SECTION – B (30 Marks)

Q4 (Compulsory) – 12 Marks

Q4 (a) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define Tree

A **Tree** is a **non-linear hierarchical data structure** consisting of **nodes** with:

- One **root node**
- Zero or more **child nodes**
- No cycles

Example:

```
    10
   / \
  20  30
```

2. What is a Binary Search Tree (BST)?

A **BST** is a binary tree in which:

- Left child < Root
- Right child > Root
- Both left and right subtrees are BSTs

C++ Node Structure:

```
struct Node{
    int data;
    Node* left;
    Node* right;
};
```

3. Define Balance Factor

Balance Factor (BF) is used in **AVL trees**:

$$BF = \text{Height}(\text{Left Subtree}) - \text{Height}(\text{Right Subtree})$$

- **BF = -1, 0, +1** → Balanced
 - **BF > 1 or BF < -1** → Unbalanced
-

4. What is BFS (Breadth First Search)?

BFS traverses a graph **level by level** using a **queue**.

Algorithm:

1. Start at source node, mark visited, enqueue
 2. Dequeue node, visit all unvisited neighbors
 3. Repeat until queue is empty
-

Q4 (b) Answer ANY THREE ($3 \times 2 = 6$ Marks)

1. Define DFS (Depth First Search)

DFS explores a graph **as deep as possible** before backtracking using:

- **Stack or recursion**

Algorithm (Recursive DFS):

```
void DFS(int v, vector<int> adj[], bool visited[]){
    visited[v] = true;
    cout << v << " ";
    for(int u : adj[v]){
        if(!visited[u]) DFS(u, adj, visited);
    }
}
```

2. What is Hash Table?

A **Hash Table** stores data using a **key-value pair** with a **hash function** mapping keys to indices.

Example:

```
int hashFunction(int key){ return key % 10; }
```

3. Define Adjacency Matrix

An **Adjacency Matrix** is a 2D array $V \times V$ representing a graph:

- $\text{matrix}[i][j] = 1 \rightarrow$ Edge exists
- $\text{matrix}[i][j] = 0 \rightarrow$ No edge

Example:

0 1 0

1 0 1

0 1 0

4. What is Collision Resolution?

A **collision** occurs when two keys hash to the **same index**.

Collision Resolution techniques:

- **Chaining:** Linked list at each index
 - **Open Addressing:** Linear probing, quadratic probing
-

Q5 – 9 Marks

Q5 (a) Explain Deletion in Binary Search Tree (4 Marks)

Deletion Cases in BST:

1. **Node with no child** $\rightarrow$ Remove node
2. **Node with one child** $\rightarrow$ Replace node with child
3. **Node with two children** $\rightarrow$ Replace with **inorder successor** or **predecessor**

C++ Code:

```
Node* deleteNode(Node* root, int key){  
    if(!root) return root;
```

```

if(key < root->data) root->left = deleteNode(root->left, key);
else if(key > root->data) root->right = deleteNode(root->right, key);
else {
    // Node with one or no child
    if(!root->left){
        Node* temp = root->right;
        delete root;
        return temp;
    } else if(!root->right){
        Node* temp = root->left;
        delete root;
        return temp;
    }
    // Node with two children
    Node* temp = root->right;
    while(temp->left) temp = temp->left; // Inorder successor
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
return root;
}

```

Q5 (b) Answer ANY ONE (5 Marks)

Option 1: Deletion Cases in BST with Example

- **Case 1:** Delete leaf node 10 → Remove node
- **Case 2:** Delete node 20 with one child → Replace with child
- **Case 3:** Delete node 30 with two children → Replace with inorder successor

Diagram Example:

30
/ \
20 40
/
10

Delete 20 → Replace with 10

Option 2: Inorder Successor and Predecessor

- **Inorder Successor:** Next node in inorder traversal (smallest in right subtree)
 - **Inorder Predecessor:** Previous node in inorder traversal (largest in left subtree)
-

Q6 – 9 Marks

Q6 (a) Explain Graph Representations (4 Marks)

1. **Adjacency Matrix:** 2D array $V \times V$
 - Easy edge lookup
 - Space $O(V^2)$
2. **Adjacency List:** Array of lists for each vertex
 - Space $O(V+E)$
 - Efficient for sparse graphs

C++ Example (Adjacency List):

```
int V = 5;  
vector<int> adj[V];  
adj[0].push_back(1);  
adj[0].push_back(2);
```

Q6 (b) Answer ANY ONE (5 Marks)

Option 1: BFS Traversal with Example

Graph:

0 → 1,2

1 → 2

2 → 0,3

3 → 3

BFS Traversal (Starting from 0):

Queue: 0

Visit 0 → enqueue 1,2

Visit 1 → enqueue 2

Visit 2 → enqueue 3

Visit 3

BFS Order: 0 1 2 3

C++ Code:

```
#include <iostream>
```

```
#include <queue>
```

```
#include <vector>
```

```
using namespace std;
```

```
void BFS(int s, vector<int> adj[], int V){
```

```
    vector<bool> visited(V,false);
```

```
    queue<int> q;
```

```
    visited[s]=true;
```

```
    q.push(s);
```

```
    while(!q.empty()){
```

```
        int u = q.front(); q.pop();
```

```
        cout << u << " ";
```

```
        for(int v: adj[u]){
```

```
            if(!visited[v]){
```

```
        visited[v]=true;
        q.push(v);
    }
}
}
}
```

Option 2: DFS Traversal (Recursive)

```
void DFS(int v, vector<int> adj[], vector<bool> &visited){
    visited[v]=true;
    cout << v << " ";
    for(int u : adj[v]){
        if(!visited[u]) DFS(u, adj, visited);
    }
}
```

✅ Summary:

- Complete answers include **definitions, diagrams, examples, and C++ code**.
- Covers **all core concepts for BST deletion, graph traversal, and representations**.

COMMON QUESTIONS FROM PAPERS 1–5

SECTION A – Common Questions

1. Define Algorithm

Answer:

An **Algorithm** is a **step-by-step procedure** to solve a problem in **finite time**.

Properties:

- Finite
- Correct
- Well-defined

C++ Example:

```
int max(int a, int b){  
    if(a > b) return a;  
    else return b;  
}
```

2. Define Stack

A **Stack** is a **linear data structure** following **LIFO** (Last In First Out).

Operations:

- push() → add element
- pop() → remove element
- peek() → top element
- isEmpty() → check empty

C++ Example:

```
#include <stack>  
  
stack<int> s;  
  
s.push(10);  
s.push(20);  
s.pop();
```



```
int top = s.top();
```

3. Define Queue / Circular Queue / Priority Queue

A **Queue** is **FIFO** (First In First Out).

- **Circular Queue:** Last position connects to first → better memory utilization
- **Priority Queue:** Elements have priorities → highest priority served first

C++ Example (Queue):

```
#include <queue>

queue<int> q;

q.push(10);

q.push(20);

q.pop();
```

4. Define Linked List

A **Linked List** is a **linear data structure** where elements are stored in **nodes** connected by pointers.

C++ Structure:

```
struct Node{

    int data;

    Node* next;

};
```

5. Define Binary Tree / BST / AVL Tree

- **Binary Tree:** Node with at most 2 children
- **BST:** Left < Root < Right
- **AVL Tree:** Self-balancing BST with **Balance Factor = Height(left) – Height(right)**

C++ Node Example:

```
struct Node{
```

```
int data;  
Node* left;  
Node* right;  
};
```

6. Define Graph / DFS / BFS

- **Graph:** Set of vertices and edges
- **DFS:** Explore as deep as possible (stack/recursion)
- **BFS:** Explore level by level (queue)

C++ DFS Example:

```
void DFS(int v, vector<int> adj[], vector<bool> &visited){  
    visited[v] = true;  
    cout << v << " ";  
    for(int u : adj[v])  
        if(!visited[u]) DFS(u, adj, visited);  
}
```

C++ BFS Example:

```
void BFS(int s, vector<int> adj[], int V){  
    vector<bool> visited(V,false);  
    queue<int> q;  
    visited[s]=true;  
    q.push(s);  
    while(!q.empty()){  
        int u = q.front(); q.pop();  
        cout << u << " ";  
        for(int v: adj[u]){  
            if(!visited[v]){  
                visited[v]=true;
```

```
        q.push(v);
    }
}
}
```

7. Define Recursion / Recursion Stack

- **Recursion:** Function calling itself
- **Recursion Stack:** Stores parameters, return addresses of active calls

C++ Example (Factorial):

```
int factorial(int n){
    if(n == 0) return 1;
    return n * factorial(n-1);
}
```

8. Define Hashing / Hash Function / Collision

- **Hashing:** Map keys to index
 - **Hash Function:** $\text{hash}(\text{key}) = \text{key} \% \text{size}$
 - **Collision:** Two keys map to same index
 - **Collision Resolution:** Chaining / Open Addressing
-

9. Stack Overflow / Stack Underflow

- **Overflow:** Push into full stack
- **Underflow:** Pop from empty stack

C++ Example:

```
if(top == MAX-1) cout << "Stack Overflow";
if(top == -1) cout << "Stack Underflow";
```

10. Merge Sort / Quick Sort / Linear Search / Binary Search

- **Merge Sort:** Divide and conquer $\rightarrow O(n \log n)$
- **Quick Sort:** Partition array around pivot $\rightarrow O(n \log n)$ average
- **Linear Search:** Scan each element $\rightarrow O(n)$
- **Binary Search:** Requires sorted array $\rightarrow O(\log n)$

Merge Sort C++ Example:

```
void mergeSort(int arr[], int l, int r){  
    if(l<r){  
        int m = l+(r-l)/2;  
        mergeSort(arr,l,m);  
        mergeSort(arr,m+1,r);  
        merge(arr,l,m,r);  
    }  
}
```

SECTION B – Common Questions

1. Insertion / Deletion in Linked List

Singly Linked List:

```
Node* insertAtBeginning(Node* head, int val){  
    Node* newNode = new Node();  
    newNode->data = val;  
    newNode->next = head;  
    return newNode;  
}
```

```
Node* deleteAtBeginning(Node* head){  
    if(!head) return NULL;  
    Node* temp = head;  
    head = head->next;  
    delete temp;  
    return head;  
}
```

Doubly Linked List:

```
struct Node{  
    int data;  
    Node* prev;  
    Node* next;  
};
```

2. AVL Tree Rotations

- **LL Rotation:** Right rotation
- **RR Rotation:** Left rotation
- **LR / RL Rotation:** Combination

C++ Example (Right Rotation):

```
Node* rightRotate(Node* y){  
    Node* x = y->left;  
    Node* T2 = x->right;  
    x->right = y;  
    y->left = T2;  
    return x;  
}
```

3. BST Deletion / Inorder Successor & Predecessor

- Delete leaf / one child / two children
 - Replace with **inorder successor** (smallest in right subtree)
 - Replace with **inorder predecessor** (largest in left subtree)
-

4. Graph Representations

- **Adjacency Matrix:** 2D array $V \times V$
 - **Adjacency List:** Array of lists, $O(V+E)$ memory
-

5. BFS / DFS Traversal

- BFS: Queue $\rightarrow$ level order
 - DFS: Stack or recursion $\rightarrow$ deep first
-

6. Binary Tree Traversals

- **Inorder:** Left $\rightarrow$ Root $\rightarrow$ Right
- **Preorder:** Root $\rightarrow$ Left $\rightarrow$ Right
- **Postorder:** Left $\rightarrow$ Right $\rightarrow$ Root

```
void inorder(Node* root){  
    if(!root) return;
```

```

inorder(root->left);

cout << root->data << " ";

inorder(root->right);
}

```

7. Expression / Postfix / Infix Evaluation

- **Postfix Evaluation:** Use stack to compute expressions
- **Infix → Postfix Conversion:** Use stack and precedence rules

// Postfix evaluation example

```

stack<int> s;

for(char c: exp){
    if(isdigit(c)) s.push(c-'0');
    else {
        int b=s.top(); s.pop();
        int a=s.top(); s.pop();
        if(c=='+') s.push(a+b);
        else if(c=='-') s.push(a-b);
    }
}

```

Summary

These are **the most common/repeated questions across Papers 1–5**.

- Section A: Algorithm, Stack, Queue, Linked List, Recursion, Sorting, Hashing, Graphs
- Section B: Linked List ops, AVL rotations, BST operations, Graph traversal, Tree traversal, Postfix evaluation